can be used for user identification but should be avoided for access control.

[1] In some countries, banks are being required to provide secure API access to transactions and payment services to third-party apps and services. The UK's Open Banking initiative and the European Payment Services Directive 2 (PSD2) regulations are examples, both of which mandate the use of OAuth 2.

[2] An alternative way to eliminate this risk is to ensure that any newly issued token contains only scopes that are in the token used to call the login endpoint. I'll leave this as an exercise.

[3] Projects such as SELinux (https://selinuxproject.org/page/Main_Page) and AppArmor (https://apparmor.net/) bring mandatory access controls to Linux.

[4] A possible solution to this is to dynamically register each individual instance of the application as a new client when it starts up so that each gets its own unique credentials. See chapter 12 of OAuth 2 in Action (Manning) for details.

[5] AS software that supports the OpenID Connect standard may use the path `/.well-known/openid-configuration` instead. It is recommended to check both locations.

[6] The older 302 Found status code is also often used, and there is little difference between them.

[7] There is an alternative method in which the client sends the original verifier as the challenge, but this is less secure.

[8] Recall from chapter 3 that earlier versions of TLS were called SSL, and this terminology is still widespread.

[9] As you might expect by now, there is a proposal to allow the client to indicate the resource servers it intends to access: https://tools.ietf.org/html/draft-ietf-oauth-resource-indicators-02

[10] I have proposed adding public key authenticated encryption to JOSE and JWT, but the proposal is still a draft at this stage. See https://tools.ietf.org/html/draft-madden-jose-ecdh-1pu-01